

YILDIZ TECHNICAL UNIVERSITY
OPERATING SYSTEMS PROJECT REPORT



Ahmet Enis ŞİMŞİR
19011077
en.simsir@std.yildiz.edu.tr

General Information

This project has been developed using the VS Code application on the Ubuntu 20.04 operating system. Another Ubuntu computer is required to run it

Server File

The primary function of the server.c file is to store user information and also manually handle user addition processes.

```
// Kullanıcılar klasoru

char* klasor_adi = "users";

if (mkdir(klasor_adi, 0777) == 0) {
    printf("users folder is created :)\n");
} else {
    printf("There is a problem with creating folder :(\n");
}

User users[MAX_CLIENTS];
int userCount = 0;

// Örnek kullanıcı ekleme
addUser(users, userCount, 1, "ahmet", "5553335");
userCount++;
addUser(users, userCount, 2, "enis", "1212121");
userCount++;
addUser(users, userCount, 3, "simsir", "77777");
userCount++;

// Kullanıcıları dosyaya kaydetme işlemi
saveUsersToFile(users, userCount);

printf("Kullanıcılar dosyaya kaydedildi.\n");
```

As seen in the code snippet above, firstly, a folder named 'users' is created. Later on, a separate folder will be created for each user within this directory.

Later on, users are added using the 'addUser' function. The purpose of this function is to create necessary folders and files for each user.

Then the 'saveUsersToFile' function comes into play, creating the 'users.txt' file where all user information is stored.

Following this code snippet, a while loop is initiated that will run until the server is shut down. Within this loop, it checks whether a new client has connected or not. The code responsible for this check is as follows:

```
if (FD_ISSET(server_fd, &readfds)) {
    if ((new_socket = accept(server_fd, (struct sockaddr *)&address, (socklen_t *)&addrlen)) < 0) {
        perror("Acceptance failed");
        exit(EXIT_FAILURE);
    }

    printf("New client connected. Socket fd is %d, IP is : %s, Port : %d\n",
        new_socket, inet_ntoa(address.sin_addr), ntohs(address.sin_port));

    for (i = 0; i < max_clients; i++) {
        if (client_sockets[i] == 0) {
            client_sockets[i] = new_socket;
            break;
        }
    }
}
```

It can be observed that there's an 'if' statement checking for any incoming connection request on the main listening socket, 'server_fd.' If there is a request, it's checked using “`accept(server_fd, (struct sockaddr *)&address, (socklen_t *)&addrlen)`” and if the connection is successful, it's acknowledged with an output to the terminal screen.

It checks whether the maximum user limit has been exceeded. If it hasn't, the client is added to the list containing all clients.

Now, let's check the codes for client processes like sending messages or adding friends.

```
for (i = 0; i < max_clients; i++) {
    sd = client_sockets[i];

    if (FD_ISSET(sd, &readfds)) {
        if (read(sd, buffer, BUFFER_SIZE) == 0) {
            // printf("\nConnection problem!");
        } else {
            int request_type;
            char request[BUFFER_SIZE];
            sscanf(buffer, "%d %[^\\n]", &request_type, request);
        }
    }
}
```

This code block is so important. Firstly we get the buffer from the client. The buffers in my app are separated like this:

Buffer

Type Of The Process	The Client ID	Special Informations About The Process
---------------------------	---------------------	---

As seen here, buffer is just a string with necessary information for the process. For example if the client with id 1 is adding the client with id 2 as a friend, the buffer will be "4 1 2". 4 is for the type of the process (adding a new friend), 1 is for id of the person who is adding and 2 is for the person who is getting added.

After getting the buffer, sscanf function separates it as request_type and request.

If The Request Type is 1:

```
if(request_type==1){
    // Alınan mesajın hedef kullanıcı ID'sini ve içeriğini ayırma işlemi
    int target_user_id;
    char message[BUFFER_SIZE];
    sscanf(request, "%d %[^\\n]", &target_user_id, message);

    // İlgili hedef kullanıcıya mesajı iletmek için döngü
    for (j = 0; j < max_clients; j++) {
        if (client_sockets[j] != 0 && j + 1 == target_user_id) {
            send(client_sockets[j], message, strlen(message), 0);

            // Mesaj gönderen istemciye bilgi gönderme
            char delivery_message[BUFFER_SIZE];
            sprintf(delivery_message, "Message delivered to user %d", target_user_id);
            send(sd, delivery_message, strlen(delivery_message), 0);

            break;
        }
    }
}
```

That means the user asks to send a message. So the user types the ID of the person he/she wants to send a message to, and the message he/she wants to send.

Since, the buffer was separated as request_type and request before, the request has the ID of the message receiver and the message also.

As it's seen in the code, firstly the ID and the message gets separated and the message gets sent to the socket of the right client thanks to the client_sockets array.

Then the "Message delivered to user" string gets sent to the message sender.

If The Request Type is 3:

```
} else if(request_type==3){
    char dosya_yolu[50] = "users/";
    char friend_list[100];
    dosya_yolu[6] = request[0];
    strcat(dosya_yolu, "/friends.txt");
    FILE *dosya = fopen(dosya_yolu, "r"); // Dosyayı okuma modunda açma

    if (dosya != NULL) {
        char satir[100]; // Okunacak satırı tutacak karakter dizisi
        // Dosya sonuna kadar satır satır okuma ve ekrana yazdırma
        while (fgets(satir, sizeof(satir), dosya) != NULL) {
            strcat(friend_list, satir);
            strcat(friend_list, " ");
        }

        fclose(dosya); // Dosyayı kapatma
        send(sd, friend_list, strlen(friend_list), 0);
    } else {
        printf("Dosya acilamadi!\n");
    }
}
```

That means the user asks to list all his/her friends. To do that, we firstly need the location of the friends file which was created in the beginning of the code. The friends file is inside the personal folder of the user. So, we need the ID of the user. This is why the request string here only has the client ID.

As you can see, the variable 'dosya_yolu' contains the correct path to the 'friends.txt' file using the ID from the request string. Subsequently, the data from the file is sent back to the user.

If The Request Type is 4:

```

} else if(request_type==4){
    char dosya_yolu[50] = "users/";
    char friend_list[100];
    dosya_yolu[6] = request[0];
    strcat(dosya_yolu, "/friends.txt");

    FILE *dosya;
    dosya = fopen(dosya_yolu, "a"); // "a" modu dosyayı ekleme modunda açar

    if (dosya == NULL) {
        printf("Dosya acma hatasi!");
        return 1; // Hata durumunda programdan çık
    }
    // Dosyaya yeni satırı ekle
    fprintf(dosya, "%c\n", request[2]);

    // Dosyayı kapat
    fclose(dosya);
    send(sd, "Kullanici eklendi", strlen("Kullanici eklendi"), 0);
}

```

That means the user asks to add a new friend. So, the request string has the id of the user who is adding a friend and the ID of the user who is getting added as a friend.

So, the true friends.txt file gets found and then, the new ID gets added.

Client File

Running this file from the terminal requires providing the user ID ('./client 1' for instance). Therefore, checking the argc and argv values is crucial for us. Initially, we obtain the user ID by verifying these values

```

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Usage: %s <client_id>\n", argv[0]);
        return -1;
    }

    int client_id = atoi(argv[1]);
}

```

Then, we proceed with the connection process to the server. If successful, both the server terminal and the user terminal will receive an acknowledgment message

Following that, the user is prompted to select an operation: “**printf("Choose what you wanna do:\n1: Send a message\n2: Read the messages\n3: List the friends\n4: Add a new friend\n");**”.

If The Client Chooses Send A Message Operation:

```
if(choose==1) {
    // Kullanıcıdan mesaj alıp sunucuya gönderme işlemi
    printf("Enter target user ID and message: ");
    int target_user_id;
    char message[BUFFER_SIZE];
    char combined_message[BUFFER_SIZE];

    if (scanf("%d %[^\\n]", &target_user_id, message) != 2) {
        printf("Invalid input.\\n");
        return 1;
    }

    sprintf(combined_message, "%d %s", target_user_id, message);

    printf("\\ncombined message: %s", combined_message);

    char all_request[BUFFER_SIZE + 5];

    if (strlen(combined_message) + 1 > BUFFER_SIZE) {
        printf("Combined message too long.\\n");
        return 1;
    }

    strcpy(all_request, "1 ");
    strcat(all_request, combined_message);

    send(sock, all_request, strlen(all_request), 0);

    // Sunucudan gelen cevabı okuma işlemi
    char server_response[BUFFER_SIZE];
    read(sock, server_response, BUFFER_SIZE);
    printf("\\nServer: %s\\n", server_response);
}
```

After obtaining the ID and the message information from the user input, this text is combined with operation id (which is 1 for sending a message) and it is sent to the main server using the 'send' function. Subsequently, the 'read' function is employed to transfer whether the message was delivered or not into the 'server_response' string, which is then displayed on the screen.

If The Client Chooses Read Your Messages Operation:

```
else if(choose==2){  
    // Sunucudan gelen cevabı okuma işlemi  
    read(sock, buffer, BUFFER_SIZE);  
    printf("Server: %s\n", buffer);  
}
```

We're using the 'read' function to retrieve the message sent by the server using 'send'.

If The Client Chooses List The Friends Operation:

```
else if(choose==3){  
    char request[20];  
    strcpy(request, "3 ");  
    strcat(request, userIDString);  
    send(sock, request, strlen(request), 0);  
    //gelen listeyi okuma  
    char server_response[100];  
    read(sock, server_response, BUFFER_SIZE);  
    printf("\nFriend List:\n %s\n", server_response);  
}
```

The operation id (3) and the client id get added to the **request** string. Then **request** gets sent to the server and the **server_response** (the list of the friends) prints on the terminal.

If The Client Chooses Add A New Friend Operation:

```
else if(choose==4){
    int new_id;
    printf("ID of the person you wanna add as friend: ");
    scanf("%d",&new_id);
    char request[20];
    strcpy(request, "4 ");
    strcat(request, userIDString);
    strcat(request, " ");
    char new_id_string[5];
    sprintf(new_id_string, "%d", new_id);
    strcat(request, new_id_string);
    printf("\nrequest = %s",request);
    fflush(stdout);
    send(sock, request, strlen(request), 0);
    //islem tamam mesaji
    char server_response[100];
    read(sock, server_response, BUFFER_SIZE);
    printf("\nServer Response: %s\n", server_response);
}
```

The user types the id of the user who he/she wants to add as a new friend. Then it gets sent to the server like the other operations. Then the server response gets read here.

Screen Shots

When the server is activated and connected via ' ./client 1' and ' ./client 2' commands on separate terminals, the server screen appears as follows

```
enis@enis-ABRA-A5-V15-1:~/Desktop/isletim$ ./server
Server is running...
There is a problem with creating folder :(
'users/1/friends.txt' file is created.
'users/2/friends.txt' file is created.
'users/3/friends.txt' file is created.
The users are saved to users.txt file.
New client connected. Socket fd is 4, IP is : 127.0.0.1, Port : 35224
New client connected. Socket fd is 5, IP is : 127.0.0.1, Port : 35230
```

Sending a message from Client 1 to Client 2 happens as follows

```
enis@enis-ABRA-A5-V15-1:~/Desktop/isletim$ ./client 1
Choose what you wanna do:
1: Send a message
2: Read the messages
3: List the friends
4: Add a new friend
1
Enter target user ID and message: 2 please come here

combined message: 2 please come here
Server: Message delivered to user 2
Choose what you wanna do:
1: Send a message
2: Read the messages
3: List the friends
4: Add a new friend
█
```

Client 2 is viewing the incoming message

```
enis@enis-ABRA-A5-V15-1:~/Desktop/isletim$ ./client 2
Choose what you wanna do:
1: Send a message
2: Read the messages
3: List the friends
4: Add a new friend
2

Server: please come here
Choose what you wanna do:
1: Send a message
2: Read the messages
3: List the friends
4: Add a new friend
█
```

Client 1 is adding a new friend and then checking the friends list:

```
enis@enis-ABRA-A5-V15-1:~/Desktop/isletim$ ./client 1
Choose what you wanna do:
1: Send a message
2: Read the messages
3: List the friends
4: Add a new friend
4
ID of the person you wanna add as friend: 2

Server Response: The user is added      s[♦
Choose what you wanna do:
1: Send a message
2: Read the messages
3: List the friends
4: Add a new friend
3

Friend List:
2
user is added s[♦
Choose what you wanna do:
1: Send a message
2: Read the messages
```

The Folders:

Name



users



client



client.c



server



server.c



users.txt